

OpenGL - Visor de Moléculas

Alumno: Josue Samuel Philco Puma

1 de julio de 2025

1. Implementación de un visor 3D de moléculas

Para esta práctica pasamos de usar OpenCV a utilizar OpenGL que está desarrollado para gráficos 2D y 3D proporcionando instrucciones necesarias para interactuar con el hardware de la tarjeta gráfica. Así que, se nos pide realizar un visor de moléculas utilizando OpenGL, la ventaja que obtenemos es que podemos utilizar todas las funciones que nos ofrece OpenGL para realizar este visor. Así que, enumeremos los pasos que debemos realizar para implementar este visor de moléculas.

1. OpenGL no es como OpenCV que se puede descargar, este ya viene integrado en la tarjeta de video por lo que requeriremos de unas librerías para realizarlo.
2. Al momento de configurar nuestro entorno debemos saber de que se va a componer nuestro visor, según el ejemplo mostrado necesitamos representar átomos junto con sus enlaces que componen una molécula.
3. Debemos definir unos shaders que serán útiles en las coordenadas de los vértices y asignación de colores.
4. Ahora para representar una molécula debemos contar con un archivo que contenga la estructura necesaria de la molécula, además de leer toda la estructura que contiene el archivo.
5. Requerimos de un modelo de buffers para convertir la información de los átomos y enlaces en listas de coordenadas tridimensionales y enviarlas a la memoria de la GPU.
6. Debemos configurar las transformaciones necesarias para la visualización 3D, como una matriz de modelo para aplicar rotaciones, una matriz de vista para simular una cámara, y una matriz de proyección para aplicar perspectiva.
7. Tener un renderizado que vaya actualizando la entrada de usuario (rotación y zoom) de forma continua.
8. Finalmente, al momento de cerrar el visor debemos liberar todos los recursos para una mejor gestión de la memoria.

Como podemos observar, se requiere muchos pasos para obtener el visor realizado, entonces vayamos paso por paso en el código para llegar al objetivo final.

```
1 #include <glad/glad.h>
2 #include <GLFW/glfw3.h>
3 #include <glm/glm.hpp>
4 #include <glm/gtc/matrix_transform.hpp>
```

Primero que nada empezamos con las librerías que usaremos que son glad para gestionar funciones de OpenGL, GLFW para crear una ventana y capturar eventos del teclado y GLM para operaciones matemáticas.

```
1 const char* vertex_shader_source = R"(
2 #version 330 core
3 layout (location = 0) in vec3 aPos;
4 uniform mat4 mvp;
5 void main() {
6     gl_Position = mvp * vec4(aPos, 1.0);
7 });
8
9 const char* fragment_shader_source = R"(
10 #version 330 core
11 out vec4 FragColor;
12 uniform vec3 color;
```

```
13 void main() {  
14     FragColor = vec4(color, 1.0);  
15 }";
```

Ahora debemos crear los shaders que vamos a utilizar para procesar los vértices, el primero transformará las coordenadas de cada átomo o enlace, y el segundo para asignar un color a los objetos renderizados.

```
1 struct Atom {  
2     float position_x, position_y, position_z;  
3     string symbol;  
4 };  
5  
6 struct Bond {  
7     int atom_1, atom_2;  
8 };  
9  
10 vector<Atom> atoms;  
11 vector<Bond> bonds;  
12 float rotation_x = 0, rotation_y = 0, zoom = 10.0f;  
13 const float rotation_speed = 0.1f, zoom_speed = 0.02f;
```

Ahora definimos las estructuras que vamos a utilizar que son para los Átomos y para los Enlaces, a la vez que requerimos de variables globales que usaremos más adelante para la interacción con el usuario.

```
1 void load_file(const string& file) {  
2     ifstream file_molecule(file);  
3     if (!file_molecule.is_open()) {  
4         cerr << "Error opening file: " << file << endl;  
5         return;  
6     }  
7  
8     string line;  
9     for (int i = 0; i < 4; i++) {  
10         getline(file_molecule, line);  
11     }  
12     int num_atoms = stoi(line.substr(0, 3));  
13     int num_bonds = stoi(line.substr(3, 3));  
14  
15     for (int i = 0; i < num_atoms; i++) {  
16         getline(file_molecule, line);  
17         Atom atom;  
18         atom.position_x = stof(line.substr(0, 10));  
19         atom.position_y = stof(line.substr(10, 10));  
20         atom.position_z = stof(line.substr(20, 10));  
21         atom.symbol = line.substr(31, 3);  
22         atoms.push_back(atom);  
23     }  
24     for (int i = 0; i < num_bonds; i++) {  
25         getline(file_molecule, line);  
26         Bond bond;  
27         bond.atom_1 = stoi(line.substr(0, 3)) - 1;  
28         bond.atom_2 = stoi(line.substr(3, 3)) - 1;  
29         bonds.push_back(bond);  
30     }  
31 }  
32  
33 unsigned int create_shader() {  
34     unsigned int vertex_shader = glCreateShader(GL_VERTEX_SHADER);  
35     glShaderSource(vertex_shader, 1, &vertex_shader_source, NULL);  
36     glCompileShader(vertex_shader);  
37  
38     unsigned int fragment_shader = glCreateShader(GL_FRAGMENT_SHADER);  
39     glShaderSource(fragment_shader, 1, &fragment_shader_source, NULL);
```

```
40     glCompileShader(fragment_shader);
41
42     unsigned int program = glCreateProgram();
43     glAttachShader(program, vertex_shader);
44     glAttachShader(program, fragment_shader);
45     glLinkProgram(program);
46
47     glDeleteShader(vertex_shader);
48     glDeleteShader(fragment_shader);
49     return program;
50 }
51
52 void atom_buffer(unsigned int& Va0, unsigned int& Vb0, const vector<Atom>&
atoms) {
53     vector<float> vertices;
54     for (const auto& atom : atoms) {
55         vertices.push_back(atom.position_x);
56         vertices.push_back(atom.position_y);
57         vertices.push_back(atom.position_z);
58     }
59     glGenVertexArrays(1, &Va0);
60     glGenBuffers(1, &Vb0);
61     glBindVertexArray(Va0);
62     glBindBuffer(GL_ARRAY_BUFFER, Vb0);
63     glBufferData(GL_ARRAY_BUFFER, vertices.size() * sizeof(float), vertices.
data(), GL_STATIC_DRAW);
64     glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 3 * sizeof(float), (void*)
0);
65     glEnableVertexAttribArray(0);
66 }
67
68 void bond_buffer(unsigned int& Va0, unsigned int& Vb0, const vector<Bond>&
bonds, const vector<Atom>& atoms) {
69     vector<float> vertices;
70     for (const auto& bond : bonds) {
71         if (bond.atom_1 < atoms.size() && bond.atom_2 < atoms.size()) {
72             const Atom& a1 = atoms[bond.atom_1];
73             const Atom& a2 = atoms[bond.atom_2];
74             vertices.push_back(a1.position_x);
75             vertices.push_back(a1.position_y);
76             vertices.push_back(a1.position_z);
77             vertices.push_back(a2.position_x);
78             vertices.push_back(a2.position_y);
79             vertices.push_back(a2.position_z);
80         }
81     }
82     glGenVertexArrays(1, &Va0);
83     glGenBuffers(1, &Vb0);
84     glBindVertexArray(Va0);
85     glBindBuffer(GL_ARRAY_BUFFER, Vb0);
86     glBufferData(GL_ARRAY_BUFFER, vertices.size() * sizeof(float), vertices.
data(), GL_STATIC_DRAW);
87     glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 3 * sizeof(float), (void*)
0);
88     glEnableVertexAttribArray(0);
89 }
90
91 void render(GLFWwindow* window, unsigned int shader, unsigned int atom_Va0,
unsigned int bond_Va0, const vector<Atom>& atoms, const vector<Bond>& bonds)
{
92     glPointSize(8.0f);
93     while (!glfwWindowShouldClose(window)) {
94         glClearColor(0.1f, 0.1f, 0.1f, 1.0f);
```

```

95     glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
96     glUseProgram(shader);
97
98     mat4 model = mat4(1.0f);
99     model = rotate(model, radians(rotation_x), vec3(1.0f, 0.0f, 0.0f));
100    model = rotate(model, radians(rotation_y), vec3(0.0f, 1.0f, 0.0f));
101    mat4 view = translate(mat4(1.0f), vec3(0.0f, 0.0f, -zoom));
102    mat4 projection = perspective(radians(45.0f), 800.0f / 600.0f, 0.1f,
        100.0f);
103    mat4 mvp = projection * view * model;
104
105    int mvp_location = glGetUniformLocation(shader, "mvp");
106    glUniformMatrix4fv(mvp_location, 1, GL_FALSE, &mvp[0][0]);
107    int color_location = glGetUniformLocation(shader, "color");
108    glUniform3f(color_location, 1.0f, 0.2f, 0.2f);
109    glBindVertexArray(atom_Va0);
110    glDrawArrays(GL_POINTS, 0, atoms.size());
111    glUniform3f(color_location, 1.0f, 1.0f, 1.0f);
112    glBindVertexArray(bond_Va0);
113    glDrawArrays(GL_LINES, 0, bonds.size() * 2);
114    glfwSwapBuffers(window);
115    glfwPollEvents();
116  }
117 }
  
```

En esta parte del código se pone más interesante, ya que estamos llegando a la parte de visualizar la molécula. Primero se carga un archivo de formato **sdf** para extraer las posiciones de los átomos y enlaces. Luego contruirá los shaders necesarios para procesar los vértices y aplicar color. Luego convierten la información estructural en buffers que serán enviados a la GPU para finalmente dibujar la molécula mostrando los átomos como puntos y los enlaces como líneas.

Ahora con esta esta parte de código veamos como es que llega a verse nuestra molécula. Para este caso estamos eligiendo la molécula de **Aspirina** para mostrarla.

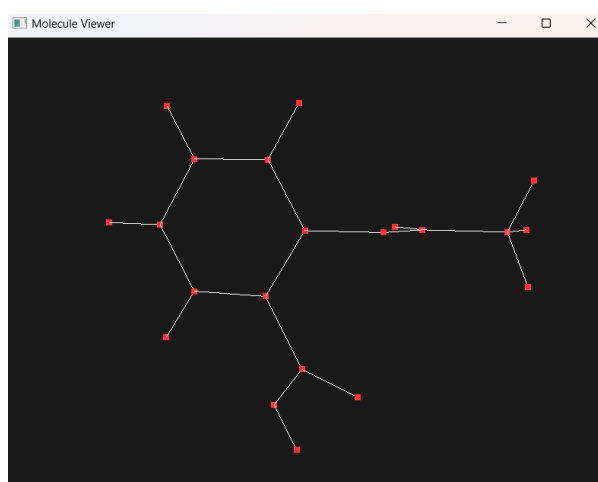


Figura 1: Mostrando la molécula en 3D

Ahora debemos tener en cuenta lo siguiente, este puede llegar a parecer una demostración de la molécula como una imagen 2D y no en 3D, aparte que se nos pide que se pueda interactuar con el visor de moléculas, así que pasemos a la siguiente parte.

2. Interacción básica para el usuario

Entonces, algo que debe permitir nuestro visor de moléculas es realizar acciones como rotación y poder hacer un zoom en la molécula, para ello debemos crear una función que permita ciertas acciones, para

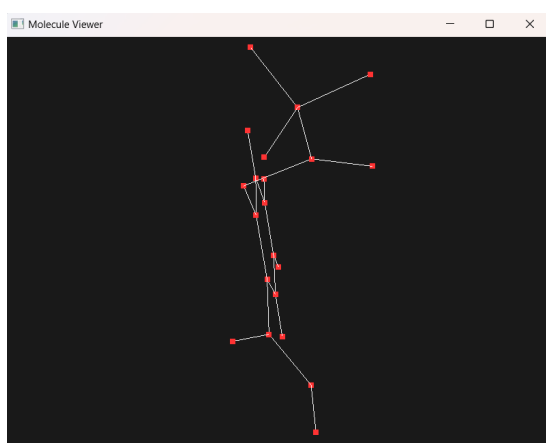
ello haremos uso de las funciones que nos permite GLFW, que es capturar eventos de nuestro teclado.

```

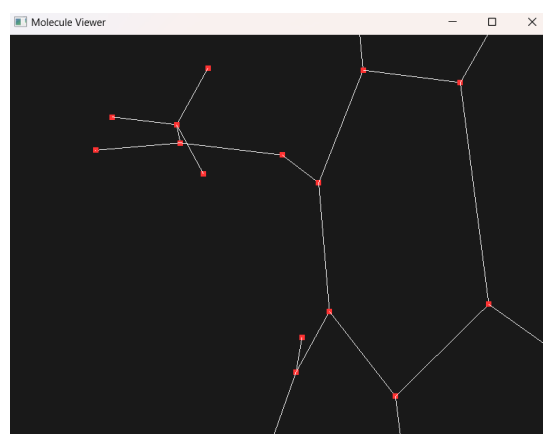
1 void interface_user(GLFWwindow* window) {
2     if (glfwGetKey(window, GLFW_KEY_ESCAPE) == GLFW_PRESS)
3         glfwSetWindowShouldClose(window, true);
4     if (glfwGetKey(window, GLFW_KEY_LEFT) == GLFW_PRESS)
5         rotation_y -= rotation_speed;
6     if (glfwGetKey(window, GLFW_KEY_RIGHT) == GLFW_PRESS)
7         rotation_y += rotation_speed;
8     if (glfwGetKey(window, GLFW_KEY_UP) == GLFW_PRESS)
9         rotation_x -= rotation_speed;
10    if (glfwGetKey(window, GLFW_KEY_DOWN) == GLFW_PRESS)
11        rotation_x += rotation_speed;
12    if (glfwGetKey(window, GLFW_KEY_W) == GLFW_PRESS)
13        zoom -= zoom_speed;
14    if (glfwGetKey(window, GLFW_KEY_S) == GLFW_PRESS)
15        zoom += zoom_speed;
16 }

```

Esta función permitirá las acciones mencionadas anteriormente, con ello ya debemos poder realizar rotación y zoom. Veamos los resultados



(a) Molécula con rotación aplicada



(b) Molécula con zoom aplicado

Figura 2: Mostrando la molécula con rotación (a) y la molécula con zoom (b)

Como se observa en la imagen, se puede observar que ya podemos realizar

3. Limitaciones y dificultades

Dentro de este laboratorio pude encontrar limitaciones y dificultades para realizarlo, de entre todos llevo a destacar las siguientes:

- Una dificultad encontré era saber que era OpenGL, al inicio pensé que era como OpenCV, es decir, algo para instalar desde internet, pero en base a tutoriales me di cuenta que ya venía integrado en nuestra máquina, solo que había que configurar bibliotecas adicionales para empezar a trabajar.
- Otra dificultad encontré es el uso de algunas funciones que para OpenGL es muy antiguo, para ser exactos, ya no se aceptan funciones utilizadas en versiones antiguas, a partir de una versión 3.x no permite utilizarlas.
- Una de las limitaciones encontré es un bajo conocimiento de OpenGL, es basarse en tutoriales para desarrollar OpenGL, en el caso del laboratorio me basé en algo simple para mostrar los átomos y enlaces, pero vi que se podía utilizar **vertex_shader_source** y **fragment_shader_source** para poder dibujar esferas, pero no me atreví por falta de información.

En resumen, este laboratorio nos llega a enseñar el poder que tiene OpenGL en la manipulación de imágenes en 3D, pero también para realizarlo es necesario ver como es que funciona las funciones que nos ofrece.

4. Código completo

Entonces una vez realizado las actividades, se muestra el código completo del laboratorio.

```
1 #include <glad/glad.h>
2 #include <GLFW/glfw3.h>
3 #include <glm/glm.hpp>
4 #include <glm/gtc/matrix_transform.hpp>
5
6 #include <iostream>
7 #include <vector>
8 #include <fstream>
9 using namespace std;
10 using namespace glm;
11
12 const char* vertex_shader_source = R"(
13 #version 330 core
14 layout (location = 0) in vec3 aPos;
15 uniform mat4 mvp;
16 void main() {
17     gl_Position = mvp * vec4(aPos, 1.0);
18 }";
19
20 const char* fragment_shader_source = R"(
21 #version 330 core
22 out vec4 FragColor;
23 uniform vec3 color;
24 void main() {
25     FragColor = vec4(color, 1.0);
26 }";
27
28 struct Atom {
29     float position_x, position_y, position_z;
30     string symbol;
31 };
32
33 struct Bond {
34     int atom_1, atom_2;
35 };
36
37 vector<Atom> atoms;
38 vector<Bond> bonds;
39 float rotation_x = 0, rotation_y = 0, zoom = 10.0f;
40 const float rotation_speed = 0.1f, zoom_speed = 0.02f;
41
42 void load_file(const string& file) {
43     ifstream file_molecule(file);
44     if (!file_molecule.is_open()) {
45         cerr << "Error opening file: " << file << endl;
46         return;
47     }
48
49     string line;
50     for (int i = 0; i < 4; i++) {
51         getline(file_molecule, line);
52     }
53     int num_atoms = stoi(line.substr(0, 3));
54     int num_bonds = stoi(line.substr(3, 3));
55 }
```

```
56     for (int i = 0; i < num_atoms; i++) {
57         getline(file_molecule, line);
58         Atom atom;
59         atom.position_x = stof(line.substr(0, 10));
60         atom.position_y = stof(line.substr(10, 10));
61         atom.position_z = stof(line.substr(20, 10));
62         atom.symbol = line.substr(31, 3);
63         atoms.push_back(atom);
64     }
65     for (int i = 0; i < num_bonds; i++) {
66         getline(file_molecule, line);
67         Bond bond;
68         bond.atom_1 = stoi(line.substr(0, 3)) - 1;
69         bond.atom_2 = stoi(line.substr(3, 3)) - 1;
70         bonds.push_back(bond);
71     }
72 }
73
74 unsigned int create_shader() {
75     unsigned int vertex_shader = glCreateShader(GL_VERTEX_SHADER);
76     glShaderSource(vertex_shader, 1, &vertex_shader_source, NULL);
77     glCompileShader(vertex_shader);
78
79     unsigned int fragment_shader = glCreateShader(GL_FRAGMENT_SHADER);
80     glShaderSource(fragment_shader, 1, &fragment_shader_source, NULL);
81     glCompileShader(fragment_shader);
82
83     unsigned int program = glCreateProgram();
84     glAttachShader(program, vertex_shader);
85     glAttachShader(program, fragment_shader);
86     glLinkProgram(program);
87
88     glDeleteShader(vertex_shader);
89     glDeleteShader(fragment_shader);
90     return program;
91 }
92
93 void interface_user(GLFWwindow* window) {
94     if (glfwGetKey(window, GLFW_KEY_ESCAPE) == GLFW_PRESS)
95         glfwSetWindowShouldClose(window, true);
96     if (glfwGetKey(window, GLFW_KEY_LEFT) == GLFW_PRESS)
97         rotation_y -= rotation_speed;
98     if (glfwGetKey(window, GLFW_KEY_RIGHT) == GLFW_PRESS)
99         rotation_y += rotation_speed;
100    if (glfwGetKey(window, GLFW_KEY_UP) == GLFW_PRESS)
101        rotation_x -= rotation_speed;
102    if (glfwGetKey(window, GLFW_KEY_DOWN) == GLFW_PRESS)
103        rotation_x += rotation_speed;
104    if (glfwGetKey(window, GLFW_KEY_W) == GLFW_PRESS)
105        zoom -= zoom_speed;
106    if (glfwGetKey(window, GLFW_KEY_S) == GLFW_PRESS)
107        zoom += zoom_speed;
108 }
109
110 void atom_buffer(unsigned int& Va0, unsigned int& Vb0, const vector<Atom>&
111 atoms) {
112     vector<float> vertices;
113     for (const auto& atom : atoms) {
114         vertices.push_back(atom.position_x);
115         vertices.push_back(atom.position_y);
116         vertices.push_back(atom.position_z);
117     }
118     glGenVertexArrays(1, &Va0);
```

```
118     glGenBuffers(1, &Vb0);
119     glBindVertexArray(Va0);
120     glBindBuffer(GL_ARRAY_BUFFER, Vb0);
121     glBufferData(GL_ARRAY_BUFFER, vertices.size() * sizeof(float), vertices.
122         data(), GL_STATIC_DRAW);
123     glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 3 * sizeof(float), (void*)
124         0);
125     glEnableVertexAttribArray(0);
126 }
127
128 void bond_buffer(unsigned int& Va0, unsigned int& Vb0, const vector<Bond>&
129     bonds, const vector<Atom>& atoms) {
130     vector<float> vertices;
131     for (const auto& bond : bonds) {
132         if (bond.atom_1 < atoms.size() && bond.atom_2 < atoms.size()) {
133             const Atom& a1 = atoms[bond.atom_1];
134             const Atom& a2 = atoms[bond.atom_2];
135             vertices.push_back(a1.position_x);
136             vertices.push_back(a1.position_y);
137             vertices.push_back(a1.position_z);
138             vertices.push_back(a2.position_x);
139             vertices.push_back(a2.position_y);
140             vertices.push_back(a2.position_z);
141         }
142     }
143     glGenVertexArrays(1, &Va0);
144     glGenBuffers(1, &Vb0);
145     glBindVertexArray(Va0);
146     glBindBuffer(GL_ARRAY_BUFFER, Vb0);
147     glBufferData(GL_ARRAY_BUFFER, vertices.size() * sizeof(float), vertices.
148         data(), GL_STATIC_DRAW);
149     glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 3 * sizeof(float), (void*)
150         0);
151     glEnableVertexAttribArray(0);
152 }
153
154 void render(GLFWwindow* window, unsigned int shader, unsigned int atom_Va0,
155     unsigned int bond_Va0, const vector<Atom>& atoms, const vector<Bond>& bonds)
156 {
157     glPointSize(8.0f);
158     while (!glfwWindowShouldClose(window)) {
159         interface_user(window);
160         glClearColor(0.1f, 0.1f, 0.1f, 1.0f);
161         glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
162         glUseProgram(shader);
163
164         mat4 model = mat4(1.0f);
165         model = rotate(model, radians(rotation_x), vec3(1.0f, 0.0f, 0.0f));
166         model = rotate(model, radians(rotation_y), vec3(0.0f, 1.0f, 0.0f));
167         mat4 view = translate(mat4(1.0f), vec3(0.0f, 0.0f, -zoom));
168         mat4 projection = perspective(radians(45.0f), 800.0f / 600.0f, 0.1f,
169             100.0f);
170         mat4 mvp = projection * view * model;
171
172         int mvp_location = glGetUniformLocation(shader, "mvp");
173         glUniformMatrix4fv(mvp_location, 1, GL_FALSE, &mvp[0][0]);
174         int color_location = glGetUniformLocation(shader, "color");
175         glUniform3f(color_location, 1.0f, 0.2f, 0.2f);
176         glBindVertexArray(atom_Va0);
177         glDrawArrays(GL_POINTS, 0, atoms.size());
178         glUniform3f(color_location, 1.0f, 1.0f, 1.0f);
179         glBindVertexArray(bond_Va0);
180         glDrawArrays(GL_LINES, 0, bonds.size() * 2);
181     }
```



```
173     glfwSwapBuffers(window);
174     glfwPollEvents();
175 }
176 }
177
178 int main() {
179     string file_molecule = "D:/UNSA EPCC/7mo semestre/Computacion Grafica/
180                             Unidad 3/Conformer3D_COMPOUND_CID_2244.sdf";
181     glfwInit();
182     glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);
183     glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);
184     glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);
185     GLFWwindow* window_view = glfwCreateWindow(800, 600, "Molecule Viewer",
186         NULL, NULL);
187     if (!window_view) {
188         cerr << "Failed to create GLFW window" << endl;
189         glfwTerminate();
190         return -1;
191     }
192     glfwMakeContextCurrent(window_view);
193     gladLoadGL();
194     glad_glViewport(0, 0, 800, 600);
195     glEnable(GL_DEPTH_TEST);
196
197     load_file(file_molecule);
198     unsigned int shader_program = create_shader();
199
200     unsigned int atom_VAO, atom_VBO, bond_VAO, bond_VBO;
201     atom_buffer(atom_VAO, atom_VBO, atoms);
202     bond_buffer(bond_VAO, bond_VBO, bonds, atoms);
203     render(window_view, shader_program, atom_VAO, bond_VAO, atoms, bonds);
204
205     glDeleteVertexArrays(1, &atom_VAO);
206     glDeleteBuffers(1, &atom_VBO);
207     glDeleteVertexArrays(1, &bond_VAO);
208     glDeleteBuffers(1, &bond_VBO);
209     glDeleteProgram(shader_program);
210
211     glfwTerminate();
212     return 0;
213 }
```

Referencias

- [1] OpenFrameworks. *GLM documentation*. Disponible en: <https://openframeworks.cc/documentation/glm/>
- [2] LearnOpenGL. *Introduction to OpenGL*. Disponible en: <https://learnopengl.com/Introduction>
- [3] LearnOpenGL. *Advanced Data*. Disponible en: <https://learnopengl.com/Advanced-OpenGL/Advanced-Data>
- [4] Pinch, M. (2020). *What are GLAD, GLFW, and OpenGL?* Disponible en: <https://matt-pinch.medium.com/what-are-glad-glfw-and-opengl-569136024c87>